

Detection Gaps for Spatial Memory Corruption in LLM-Generated C:

An Empirical Study of Intra-Object Overflow Across Detectors, Toolchains, and Code Models

Sadik Erisen*

Independent Researcher & Graduate Student
University of Pennsylvania

July 13, 2026

Abstract

Large language models increasingly generate C code that is compiled and run inside sandboxes trusted to contain untrusted behaviour, and that trust rests on a stack of commodity defences: compiler warnings, static analyzers, build-time hardening, and sanitizers. This paper measures, for the specific class of spatial memory corruption, which of those defences actually fire and whether model-generated C lands in the gaps they leave. We point ten independent commodity detectors at six memory-corruption classes and find that one class, *intra-object* (adjacent-field) overflow, is missed by every detector, including AddressSanitizer and Valgrind, because the write stays inside a single valid allocation and crosses no boundary. The gap is not a toolchain artifact: it holds across five compilers, two `_FORTIFY_SOURCE` levels, AddressSanitizer under two back-ends, and two CPU architectures. We then give three local code models 216 security-neutral fixed-buffer tasks and check every generated snippet against a single translation-unit, AddressSanitizer-backed oracle calibrated on golden-safe and golden-vulnerable references. 11 of 213 usable snippets contain a confirmed spatial overflow, concentrated in the smallest model, and the commodity static stack diagnosed the overflow in 0 of the 11. We connect the results at the detection-layer level: the shipped static layer catches none of these generated overflows, and the shipped runtime tool is blind to the intra-object subclass, so neither layer is a sufficient detector for the class. We report every number with its limits, including why the rate is a lower bound and a false positive removed by hand-verification.

Introduction

For decades, memory-safety bugs have been the largest single category of serious software vulnerabilities. Out-of-bounds write (CWE-787) and out-of-bounds read (CWE-125) sit near the top of the MITRE CWE Top 25, and vendor data has long attributed roughly two-thirds of critical vulnerabilities to memory errors. A new pressure has been added to this old problem: code produced by large language models is now compiled and executed by automated agents, often inside a sandbox that is trusted to hold the line if the generated code turns out to be unsafe.

That trust is the subject of this paper. When an LLM-driven agent compiles and runs its own C, the sandbox has to hold at the memory boundary inside the process, and the tools we

*This research was conducted independently. The views and findings expressed in this paper are entirely those of the author and do not reflect the position of any corporate employer. No data in this paper is fabricated; every figure traces to a released artifact.

rely on there are a specific, commodity stack: the compiler’s warnings, the static analyzers run in CI, the hardening flags baked into the build, and the sanitizers used in testing. We do not ask whether a model is “capable” of writing an exploit. We ask which memory-safety weaknesses survive that commodity stack, and whether the code a model actually generates falls into the surviving classes.

The question we set out to answer:

For spatial memory corruption, which commodity detectors actually fire per bug class, and does model-generated C land in the classes they miss?

What this paper contributes:

1. A per-detector coverage map of ten commodity detectors against six memory-corruption classes, identifying *intra-object* (adjacent-field) overflow as a total detection gap: missed by every static analyzer, every compiler warning set, build-time hardening, and both major sanitizers.
2. Evidence that the gap is structural rather than incidental: it holds across five compilers, two `_FORTIFY_SOURCE` levels, AddressSanitizer under two back-ends, and two CPU architectures.
3. A measured study of whether three local code models produce this class on ordinary fixed-buffer tasks, using an automated, calibrated, hand-verified ground truth, and of how much of what they produce the commodity static stack can see.
4. An honest account of the limits, including a false positive removed by hand-verification, a lower-bound rate, and the single-shot scope.

Background

Spatial memory safety

A spatial memory-safety violation reads or writes outside the bounds of an object. The two canonical forms are out-of-bounds write (CWE-787) and out-of-bounds read (CWE-125). In C, nothing in the language enforces bounds, so safety rests on programmer discipline and on the external tools examined here.

Intra-object versus boundary overflow

The distinction this paper turns on is where the overflow lands. An *allocation-boundary* overflow writes past the end of a heap or stack object; red-zone tools (AddressSanitizer) and shadow-memory tools (Valgrind) detect it because the write crosses into instrumented memory. An *intra-object* overflow writes past one struct field into an adjacent field of the *same* allocation:

```
struct rec { char name[16]; int uid; }; /* one allocation */
strcpy(r->name, attacker_input);      /* 20 bytes: name[16] overrun spills into uid */
/*
```

No allocation boundary is crossed, so boundary-based tools stay silent by construction. Field boundaries exist only at the ABI/type layer, one granularity finer than any commodity mechanical control operates at, which is the structural reason for the gap.

The commodity detection stack

We treat each of the following as an independent *detector* and ask, per class, whether it fires. Static and compile-time: `gcc -Wall -Wextra`, `gcc -fanalyzer`, gcc link-time type checking, clang warnings, the clang static analyzer, and `cpccheck`. Runtime: `_FORTIFY_SOURCE`, AddressSanitizer (ASan), UndefinedBehaviorSanitizer, and Valgrind/Memcheck.

Operational states of a run

For a build that exercises a bug on a given input, we classify the outcome by direct evidence (exit signal, emitted diagnostic, and a self-check the probe prints): **silent** (corruption confirmed, exit 0, no diagnostic; exploitable and undetected), **detected** (a defense named the error and stopped it), **crashed** (a raw fault with no diagnostic; denial of service, not detection), **prevented** (the unsafe op refused before corruption), and **clean** (the bug did not take effect). These are defined so that “missed” is evidence rather than opinion; a crash is deliberately not counted as a detection.

Related Work

AddressSanitizer [1] and Valgrind/Memcheck [2] are the red-zone and shadow-memory tools whose boundary model is precisely why intra-object overflow is invisible to them. The closest prior art on the intra-object case is ASan’s own field-padding extension [4], which inserts poisoned padding between fields to catch adjacent-field overflow, but only for C++ non-standard-layout classes, behind an experimental flag, and in no shipped build; that scoping is why the practical gap stands. `_FORTIFY_SOURCE` [3] provides the build-time object-size checks we measure at the shipped-hardening layer. A separate line of work reports that code models generate insecure code; Pearce et al. [5] found roughly 40% of Copilot completions across 89 security-relevant scenarios were vulnerable. Our contribution is not that claim but its connection to a measured, toolchain-general detector gap.

Threat Model

By “sandbox” we mean the commodity stack normally trusted to hold the line around compiled-and-executed code: the compiler and its hardening flags, the static analyzers run in CI, and the sanitizers used in testing. The adversary is not assumed to be external; the unsafe C may arrive because a model was manipulated or simply because it generated a bug. The asset at risk is process integrity at the memory boundary. A defense “holds” for a class if some detector moves the outcome out of the **silent** state. The central question is which classes remain **silent** under the shipped configuration, since a silent outcome is exploitable and undetected.

Methodology and Experimental Setup

We test three falsifiable hypotheses. The first, the *coverage hypothesis*, is that the commodity detection stack covers spatial memory corruption. The second, the *toolchain-generality hypothesis*, is that any gap the first finds is an artifact of a single compiler or version. The third, the *generated-code hypothesis*, is that model-generated C both produces the class and is caught by the static stack. Each is set up below and tested in the Results; as stated, each is falsified.

The coverage hypothesis

We point all ten detectors at each of six classes and record CAUGHT versus MISSED, saving raw per-cell evidence. Each class is a minimal probe with a self-check that prints

`outcome=corrupted|clean`. Each runtime detector is built with isolation flags so the column measures exactly one tool (canary and `_FORTIFY_SOURCE` disabled when measuring a sanitizer, and so on). A catch is recorded only when a memory-safety diagnostic names the defect (static) or the tool’s own runtime signature fires; compiler meta-noise such as “unknown warning option” is filtered so it can never count as a catch. We measure each detector in isolation rather than as a combined pass because the claim of interest is *which* layer is blind.

The toolchain-generality hypothesis

We re-run the six classes through five compilers (gcc-11, gcc-12, clang-14, clang-15, tcc), `_FORTIFY_SOURCE` levels 2 and 3, and ASan from both gcc and clang, and compare against a companion aarch64/gcc-15 environment. The gap holds if intra-object stays a total gap and no compiler or level newly catches it.

The generated-code hypothesis

We give three local models (qwen2.5-coder:1.5b, deepseek-coder:6.7b, qwen2.5-coder:7b) 216 security-neutral fixed-buffer tasks (18 tasks, 4 samples each), asking for a struct with a fixed `char` field filled from variable-length input, with no security framing. Ground truth is a *single-translation-unit* oracle: the test driver is concatenated with the model’s snippet so it binds to the model’s own struct and function, and any layout deviation becomes a compile error counted as `compile_fail` rather than silent undefined behaviour. Under ASan we sweep input lengths and apply two oracles: Oracle A (allocation-boundary overflow, via ASan) and Oracle B (an intra-object sentinel on a trailing scalar). Every one of the 18 drivers is calibrated against a golden-safe (`strncpy` plus explicit null) and golden-vulnerable (`strcpy`) reference before any model code is judged; all 18 passed. Every flagged case is hand-verified.

Environment

All experiments ran on Ubuntu 22.04.5 (x86_64, WSL2 kernel 6.18) with gcc-11.4/12.3, clang-14/15, cppcheck 2.7, valgrind 3.18.1, and tcc 0.9.27; models served locally via ollama.

Case Studies

The six classes exercised by the coverage and toolchain-generality hypotheses:

- **Intra-object out-of-bounds write (CWE-787)**. An unbounded copy overruns a `char` field into an adjacent field of the same struct. The load-bearing case.
- **Heap out-of-bounds write (CWE-787)**. The same overrun past the whole heap allocation.
- **Stack out-of-bounds write (CWE-787)**. The overrun past a stack buffer.
- **Use-after-free (CWE-416)**. A write through a pointer to freed memory.
- **Integer-overflow allocation (CWE-190)**. A size computation wraps, producing an undersized allocation later overrun.
- **ABI/type skew (CWE-843)**. Two translation units disagree on a struct’s type, so a field write lands at the wrong offset.

Results

Detector coverage

Table 1 gives the coverage. Intra-object overflow is caught by no detector, a total gap, while the heap and stack variants of the same CWE-787 are caught at runtime by ASan and Valgrind. All six compile-time detectors caught none of the six classes on these probes. Under the shipped build (`-O2 -D_FORTIFY_SOURCE=2`, default canary), five of six classes are silent; only the stack overflow is detected, by the canary. The intra-object row is the load-bearing result and is re-verified under the toolchain-generality hypothesis; the integer-overflow and ABI-skew rows summarise the recorded matrix and mark partial/absent cells conservatively.

Table 1: Detector coverage: which commodity detector catches which class. “static” abbreviates all six compile-time detectors. Shipped state is the `-O2 -D_FORTIFY_SOURCE=2` build.

class	static	FORTIFY	ASan	Valgrind	shipped state
intra-object (adjacent field)	miss	miss	miss	miss	silent
heap out-of-bounds write	miss	miss	CAUGHT	CAUGHT	silent
stack out-of-bounds write	miss	miss	CAUGHT	CAUGHT	detected (canary)
use-after-free	miss	–	CAUGHT	CAUGHT	silent
integer-overflow alloc	miss	–	partial	partial	silent
ABI/type skew	miss	–	–	–	silent

Toolchain generality

No compiler warned on intra-object at any level; the shipped state is silent under `_FORTIFY_SOURCE 2` and `3`; ASan misses it under both gcc and clang; the gap holds across two architectures and gcc versions 11, 12, and 15. One honest side-finding: gcc-12 emits a compile-time use-after-free warning where gcc-11 is silent, which is real but orthogonal to intra-object.

Generated-code study

Of 216 generations, 213 compiled against the oracle (3 were contract deviations). 11 of 213 usable snippets contain an ASan-confirmed spatial overflow, an overall rate of 5.2% (95% CI 3–9%), dominated by model size (Table 2). No member of the commodity static stack diagnosed the actual overflow in any of the 11 confirmed cases: 8 drew no diagnostic at all, and the rest drew a warning about a different defect. These are input-length-dependent writes in a function the analyzer sees in isolation, which is undecidable at compile time; only runtime ASan caught them, and ASan ships in no build here. We did not observe an ASan-invisible intra-object case in generated code: the one candidate our sentinel oracle flagged was a false positive, a function that legitimately assigns its trailing field, removed by hand-verification.

Table 2: Generated-code study: ASan-confirmed spatial-overflow rate by model, and how the commodity static stack diagnosed the confirmed cases.

model	confirmed / usable	rate	95% CI
qwen2.5-coder:1.5b	10/70	14.3%	8–24%
deepseek-coder:6.7b	1/71	1.4%	0–8%
qwen2.5-coder:7b	0/72	0.0%	0–5%
overflow diagnosed by any static tool: 0/11			
zero static diagnostics at all: 8/11			

Observations

The three results connect at the layer level rather than by asserting more than we measured. The shipped *static* layer caught none of the 11 generated overflows; the coverage and toolchain-generalality results show the shipped *runtime* tool (AddressSanitizer) is blind to the intra-object subclass; so neither layer is a sufficient detector for the class. One generated overflow (`parse_kv`) makes the boundary concrete: its ASan report shows the write reaching the allocation boundary only after passing *through* an adjacent field, so a bounded version of the same code would be the intra-object probe, invisible to ASan. A second observation cuts against intuition: in the generated-code study the smallest model was the most memory-unsafe, the reverse of a naive “bigger is always worse” expectation, so memory-safety carefulness is its own axis.

Recommendations

A pipeline that compiles and runs LLM-generated C should not treat a clean compile, clean static analysis, or even a clean AddressSanitizer run as evidence that adjacent-field overflow is absent. Detecting this class needs an oracle that reasons about struct-field boundaries rather than allocation boundaries: ASan field-padding on recompiled structs, a bounds-checking build, or hardware sub-object bounds where available. Where none is available, the safest posture is to treat fixed-size struct fields filled from variable-length input as a review target in generated code, since that is the exact shape the detectors miss.

Limitations

The generated-code rate is a lower bound: the intra-object oracle cannot see mid-struct overflows, the multi-field tasks are judged by the boundary oracle alone, and one excluded compile-fail contained an unbounded copy, all of which can only raise the true rate. The oracle’s intra-object check produced one false positive (a function that legitimately writes its trailing scalar), caught by hand-verification; the confirmed rate rests on AddressSanitizer. The rate is conditional on fixed-buffer tasks and on the provided interface, is measured on three small open-weight models on one machine, and reflects single-shot generation rather than a multi-turn agent. One toolchain per version was used; the intra-object result is structural and the toolchain-generalality result spans two architectures, but the broader matrix should be re-run elsewhere. A non-default ASan build with intra-object red zones can catch some of these overflows but is C++-oriented, opt-in, and part of no shipped build, so the practical gap stands.

Conclusion

Spatial memory corruption has a specific, reproducible blind spot, intra-object overflow, that no commodity detector catches and that survives five compilers, two hardening levels, both sanitizer back-ends, and two architectures. Small code models produce spatial overflows on ordinary fixed-buffer tasks at a measurable rate, and the shipped static layer diagnosed none of them. Read at the layer level, the static layer misses input-dependent overflow in isolated functions and the runtime layer misses the intra-object subclass, so the default stack is not a sufficient detector for this class. Closing the gap requires field-boundary reasoning that is not in the default stack today.

Acknowledgments and Disclosure

This research was conducted independently, on the author’s own hardware, using only local models. No figure in this paper is copied from another source or fabricated; every number traces

to a released artifact, and two honest corrections (a removed false positive and a harness bug caught by refusing an anomalous result) are reported rather than hidden.

Reproducibility

The detector matrix (`detector_matrix.py`), toolchain sweep (`h2_toolchain.py`), generated-code harness (`h3_full.py`), all `*_results.jsonl` data, the generated snippets, and the calibration references are released with the paper. Host: Ubuntu 22.04.5, x86_64, WSL2 kernel 6.18; gcc-11.4/12.3, clang-14/15, cppcheck 2.7, valgrind 3.18.1, tcc 0.9.27; models via ollama (qwen2.5-coder:1.5b/7b, deepseek-coder:6.7b).

References

- [1] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov. “AddressSanitizer: A Fast Address Sanity Checker.” *2012 USENIX Annual Technical Conference (USENIX ATC '12)*, pp. 309–318, 2012.
- [2] N. Nethercote and J. Seward. “Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation.” *Proc. 28th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI '07)*, pp. 89–100, 2007.
- [3] J. Jelinek. “Object size checking to prevent (some) buffer overflows.” GCC patches mailing list, Sept. 2004. <https://gcc.gnu.org/legacy-ml/gcc-patches/2004-09/msg02055.html>.
- [4] Google Sanitizers project. “AddressSanitizerIntraObjectOverflow.” <https://github.com/google/sanitizers/wiki/AddressSanitizerIntraObjectOverflow>.
- [5] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri. “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions.” *2022 IEEE Symposium on Security and Privacy (SP)*, pp. 754–768, 2022.